

Common Optimizations and Patterns in CUDA

Parallel Computing

Topics:

- output interference / data races
- atomics and privatization
- more on control divergence
- thread coarsening

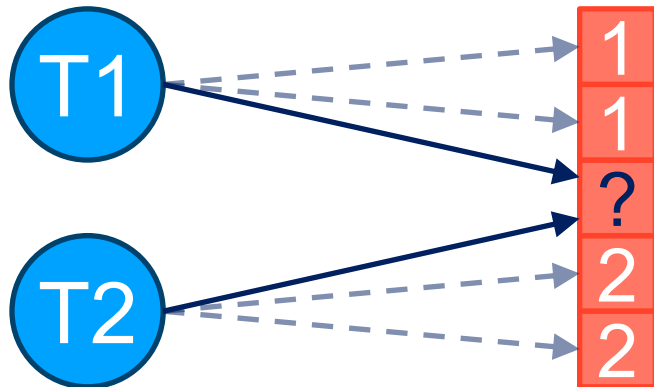


**POLITECNICO
MILANO 1863**

DIPARTIMENTO DI ELETTRONICA
INFORMAZIONE E BIOINGEGNERIA

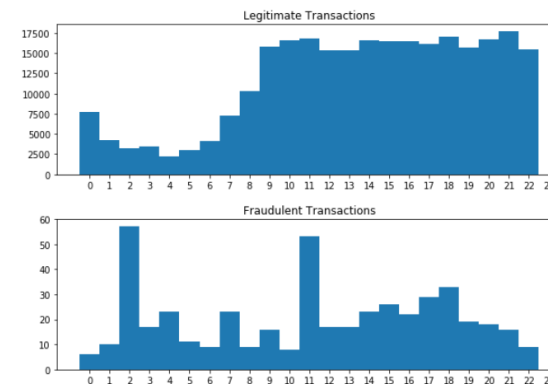
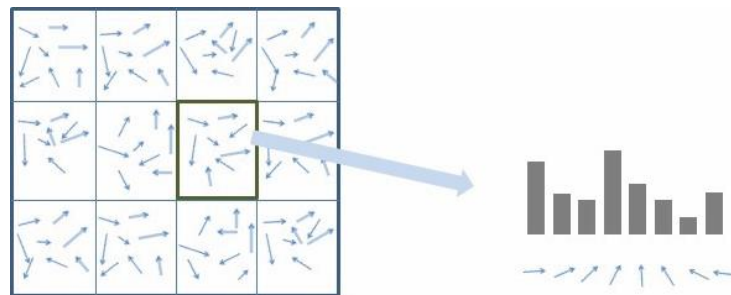
Histogram

- ❑ Computing an histogram exhibits a common and useful computation pattern
- ❑ Allows us to illustrate the concepts of
 - Partitioning
 - Atomic operations
 - Privatization
- ❑ Key issue to solve: **output interference**



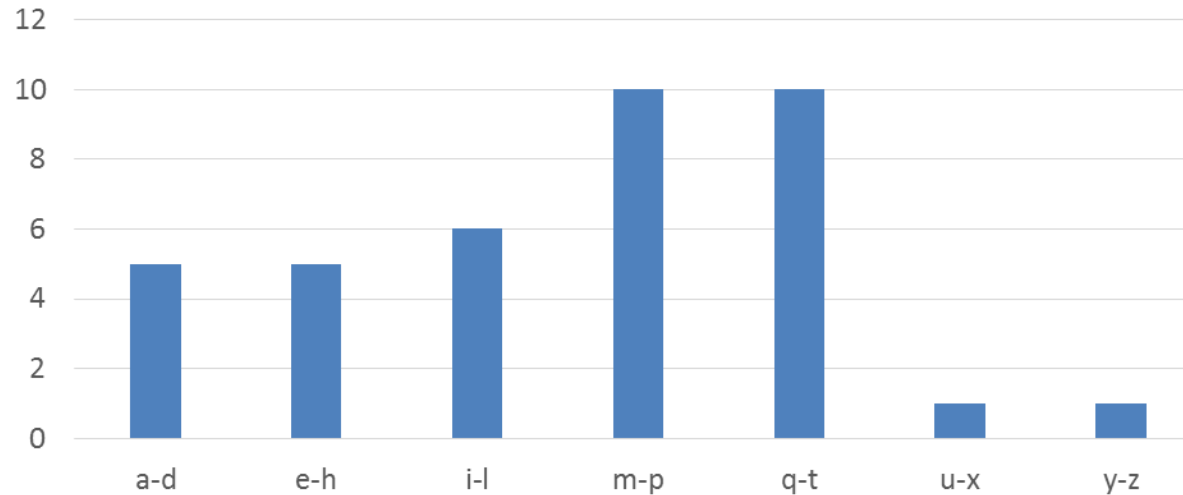
Histogram Idea

- ❑ The histogram is a method for extracting notable features and patterns from large datasets. E.g.:
 - Feature extraction in images, graphs, etc...
 - Probability density function estimation
- ❑ Idea: count instances of individual values or groups/ranges of values (called “bins”) in a dataset
- ❑ How: for each element in the dataset, use its value to identify a “bin counter” to increment



Histogram Example

- ❑ Counts the occurrence of letters in a string (grouped in bins of 4 contiguous letters)
 - For each character in the input string, increment the appropriate bin counter
- ❑ Example: histogram for the string “programming massively parallel processors”



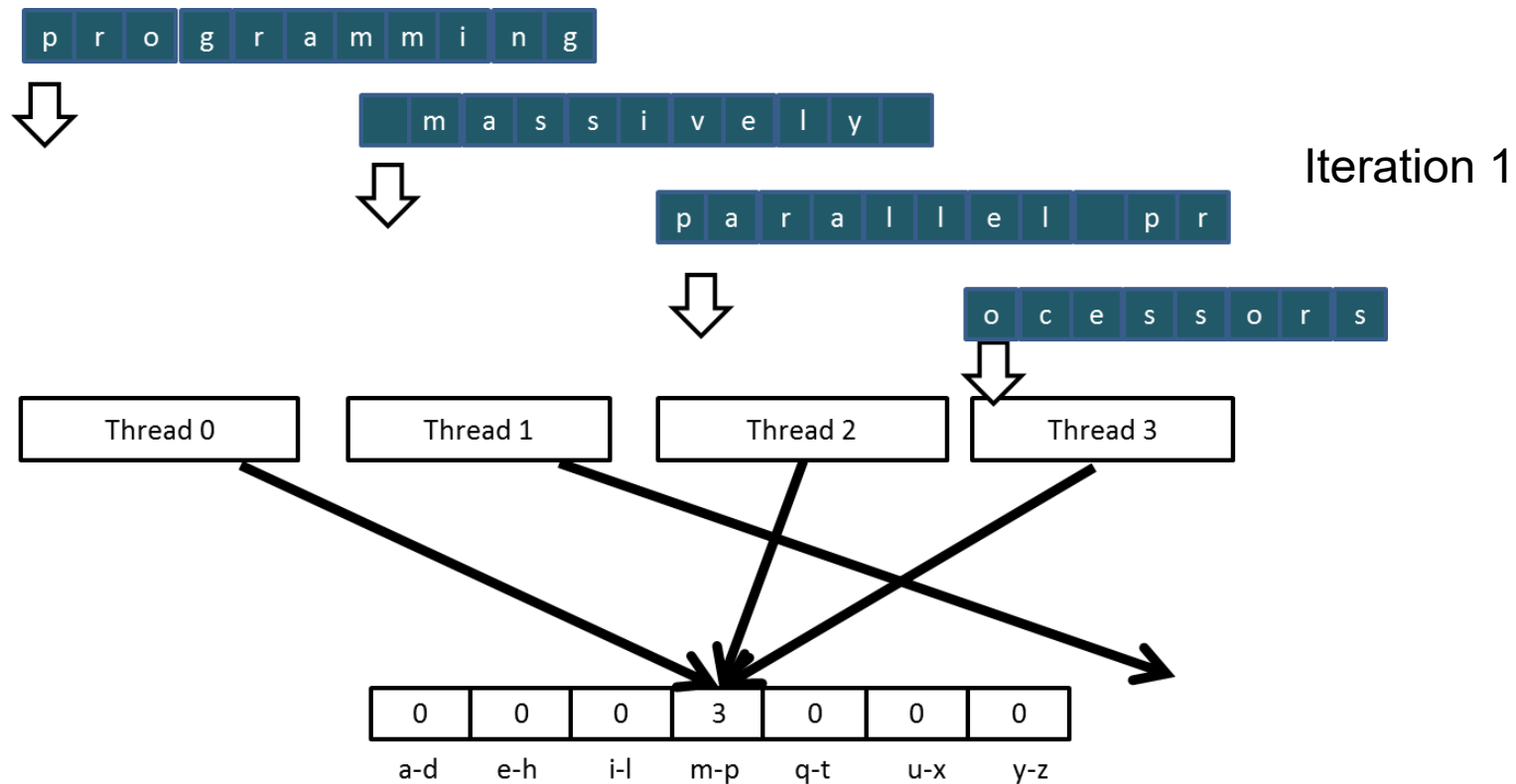
Histogram Example Code

❑ Sequential implementation in C

```
sequential_histogram(char *data, int length, int *hist) {  
    for (int i = 0; i < length; i++) {  
        int alphabet_position = data[i] - 'a';  
        if (alphabet_position >= 0 && alphabet_position < 26) {  
            hist[alphabet_position/4]++;  
        }  
    }  
}
```

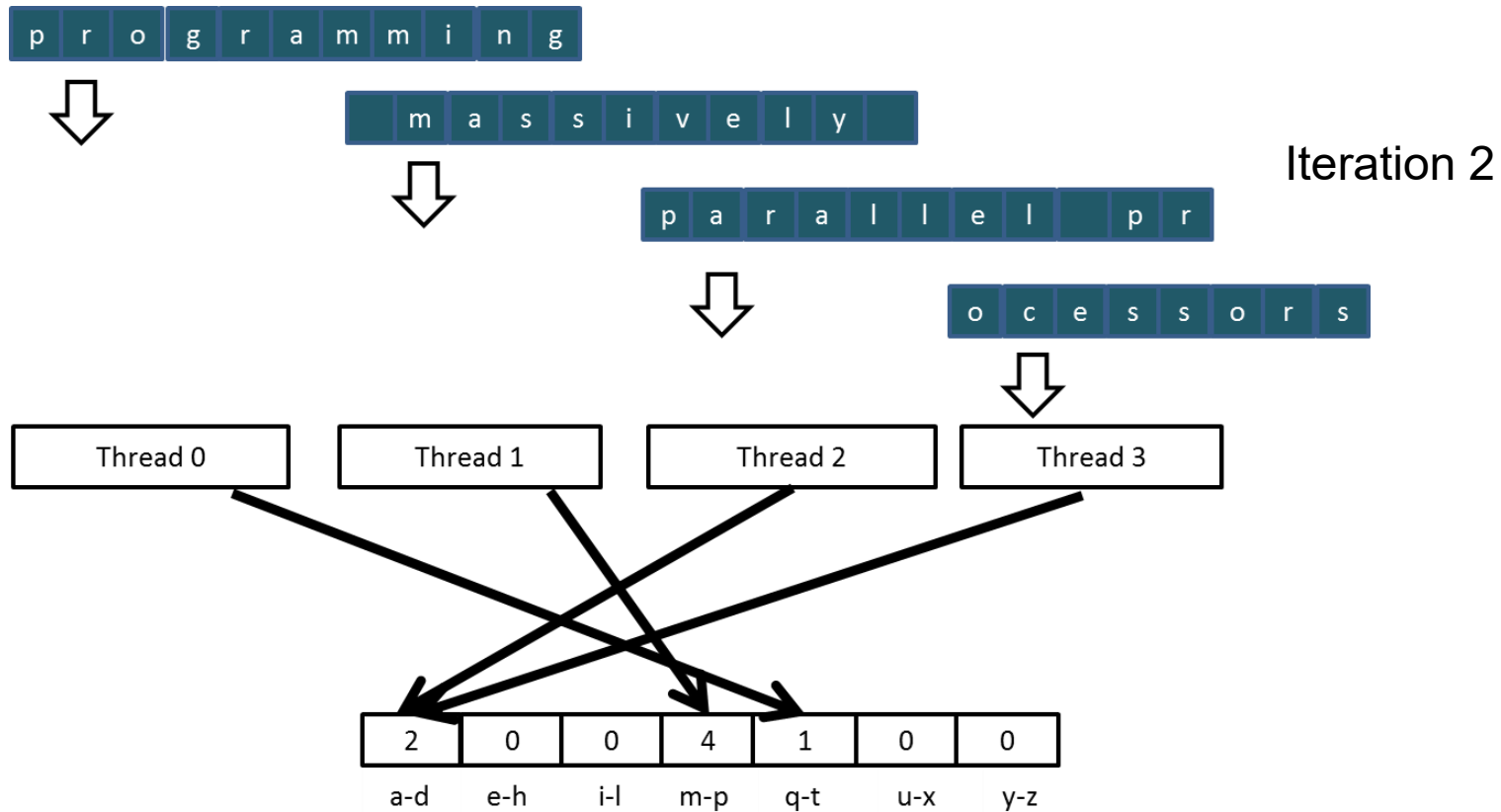
Simple Parallel Histogram

- Simplest parallel version:
 - Partition the input into sections
 - Each thread iterates through one section



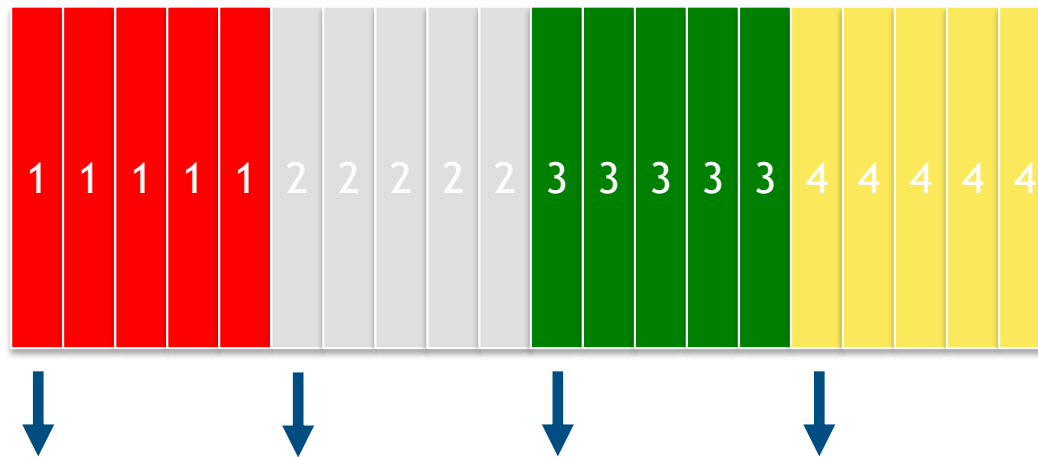
Simple Parallel Histogram

- Simplest parallel version:
 - Partition the input into sections
 - Each thread iterates through one section



Simple Parallel Histogram

- ❑ Issue: input partitioning affects memory access efficiency
 - Adjacent threads do not access adjacent memory locations
 - Accesses are not coalesced
 - DRAM bandwidth is poorly utilized



For the 1st iteration, those are the accessed value: no coalescing...

Better Data Partitioning

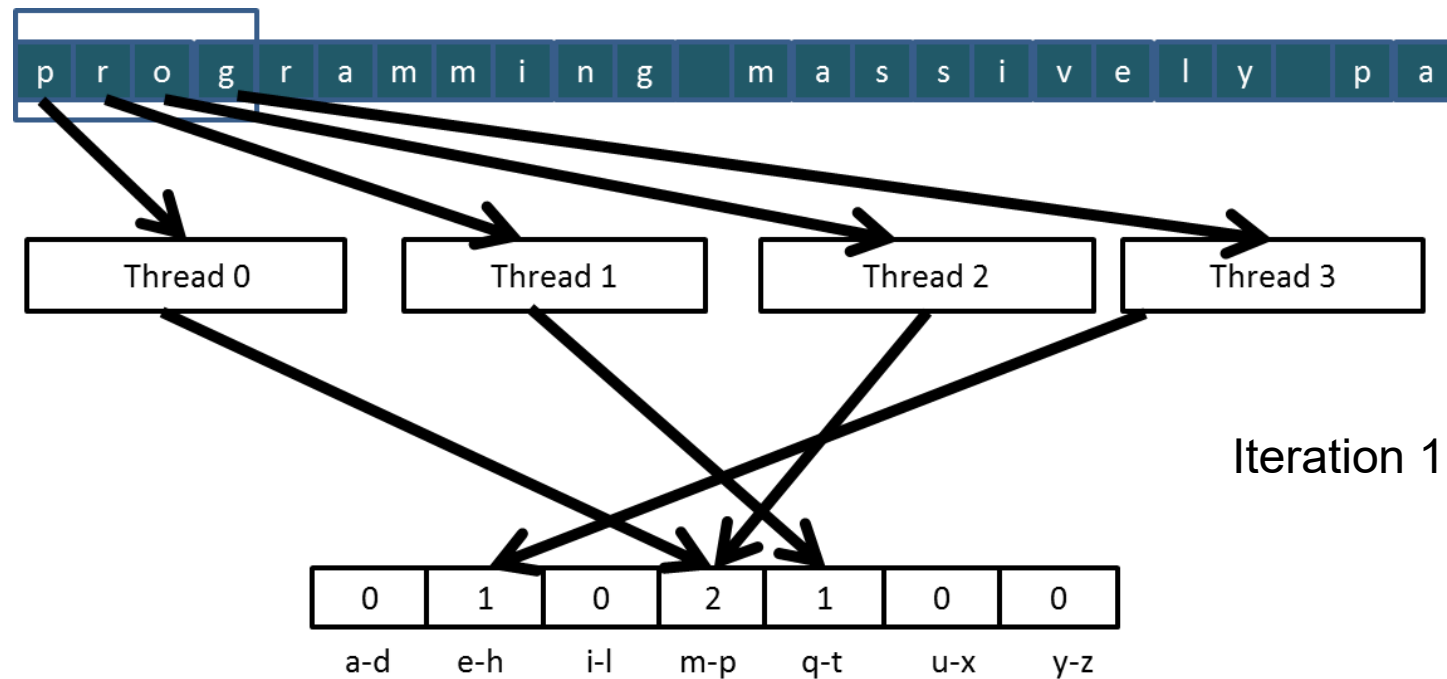
- ❑ Change to interleaved partitioning
 - At each iteration, neighboring threads process a contiguous section of elements together
 - They all move to the next section and repeat
 - Memory accesses are coalesced



Contiguous accesses can now be coalesced!

Better Data Partitioning

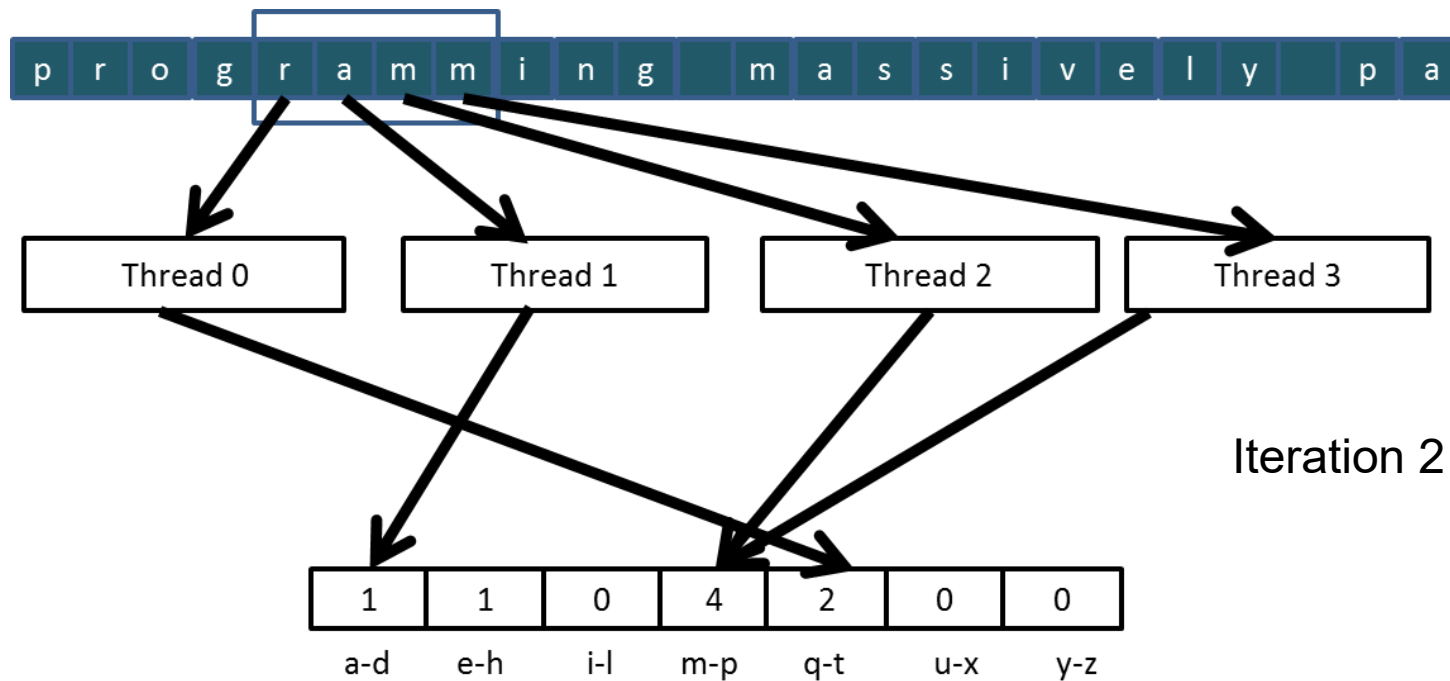
- Parallel histogram with interleaved partitioning



The whole grid behaves like a “sliding window” over the input string...

Better Data Partitioning

- Parallel histogram with interleaved partitioning



The whole grid behaves like a “sliding window” over the input string...

Better Data Partitioning

□ Parallel histogram in CUDA with interleaved partitioning

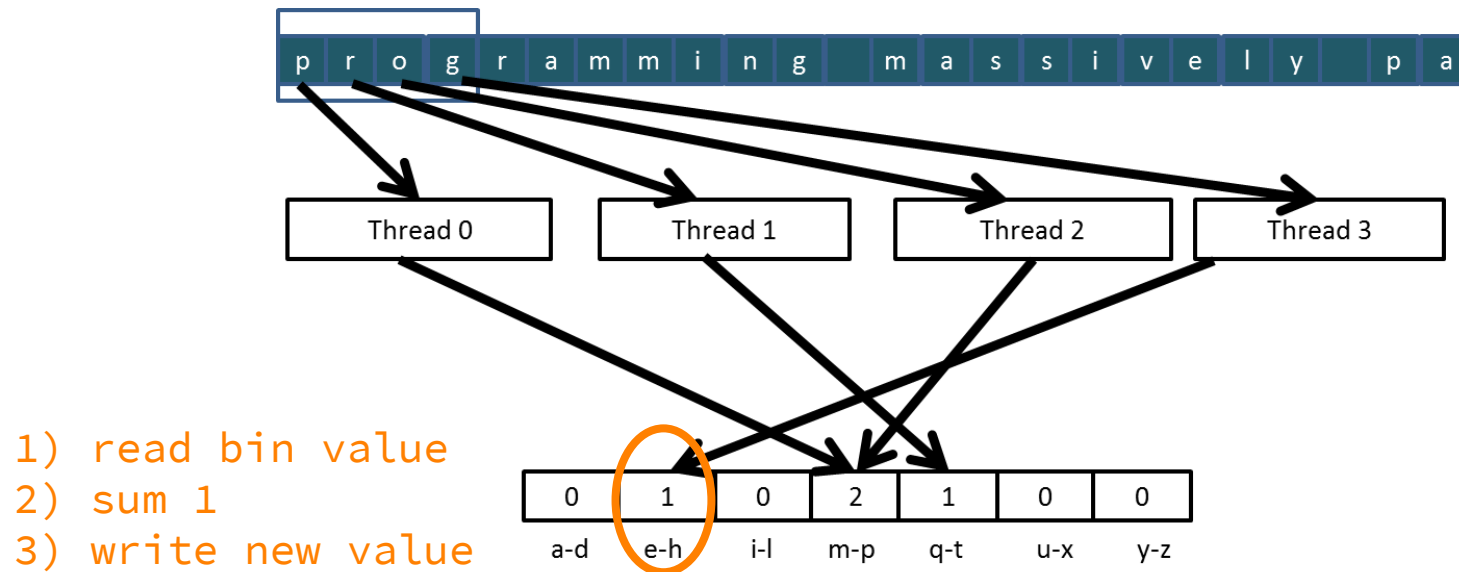
```
__global__ void hist_kernel(unsigned char *buffer, long size,
                           unsigned int *hist)
{
    int tid = threadIdx.x + blockIdx.x * blockDim.x;
    int stride = blockDim.x * gridDim.x;

    for (unsigned int i = tid; i < size; i += stride ) {
        int alphabet_position = buffer[i] - 'a';
        if (alphabet_position >= 0 && alphabet_position < 26)
            hist[alphabet_position/4] += 1;
    }
}
```

The Previous Histogram Code is BROKEN!

❑ Issue: data races

- Data races occur in **read-modify-write** operations, when multiple threads **update the same memory location**
- Data races cause errors that are not deterministic
- Histograms (and other collaborative patterns) include read-modify-write operations



Data Races

Thread 1:

```
old      = mem[x]  ---read---  
new      = old + 1 --modify--  
mem[x]   = new     --write---
```

Thread 2:

```
old      = mem[x]  
new      = old + 1  
mem[x]   = new
```

□ Data race:

- Assume x to be the same in both threads.
- If $\text{mem}[x]$ was initially 0, what is its value when threads 1 and 2 have completed?
- What does each thread get in its `old` variable?
- The answers depend on the relative execution order!

Data Races

Time	Thread 1	Thread 2
1	(0) old = mem[x]	
2	(1) new = old + 1	
3		(0) old = mem[x]
4	(1) mem[x] = new	
5		(1) new = old + 1
6		(1) mem[x] = new

Time	Thread 1	Thread 2
1		(0) old = mem[x]
2	(0) old = mem[x]	
3		(1) new = old + 1
4		(1) mem[x] = new
5	(1) new = old + 1	
6	(1) mem[x] = new	

Wrong result

Time	Thread 1	Thread 2
1	(0) old = mem[x]	
2	(1) new = old + 1	
3	(1) mem[x] = new	
4		(1) old = mem[x]
5		(2) new = old + 1
6		(2) mem[x] = new

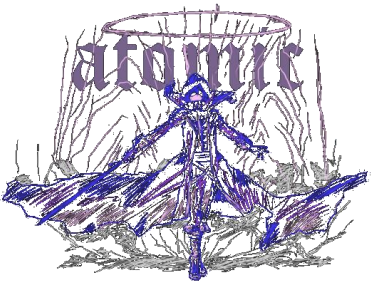
Time	Thread 1	Thread 2
1		(0) old = mem[x]
2		(1) new = old + 1
3		(1) mem[x] = new
4	(1) old = mem[x]	
5	(2) new = old + 1	
6	(2) mem[x] = new	

Correct result

❑ Fix: from read to write, all three operations must be done atomically

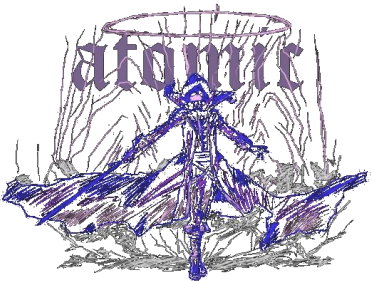
Atomic Operations

- ❑ Atomic operations perform read-modify-write as a single hardware instruction on a memory address
 - The hardware ensures that no other thread can perform another read-modify-write operation on the same location until the current atomic operation is complete
 - Any other threads that attempt to perform an atomic operation on the same location will be held in a queue
 - The execution of all threads performing an atomic operation on the same location is **serialized**



Atomic Operations

- ❑ Atomic operations in CUDA are performed by calling functions that are translated into single instructions (*intrinsic functions* or *intrinsics*)
 - available intrinsics for atomic add, sub, inc, dec, min, max, exch (exchange), CAS (compare and swap)
- ❑ Example: atomic add
 - `int atomicAdd(int* address, int val);`
 - reads a 32-bit word from address, computes `read_value + val`, and stores the result back to memory at the same address, returns the read value
 - the three operations are performed in one atomic transaction



Note: variants exist for 16-bit and 64-bit words too

Atomic Operations

❑ Example: atomic CAS

- any other atomic can be manually built with a CAS
- `int atomicCAS(int* address, int compare, int val);`
- reads a 32-bit word from `address`, compares it with `compare`, if the two coincide, writes `val` to the same memory address, returns the read value
- atomic add done with CAS:

```
do {  
    old = mem[x];  
    new = old + val;  
    seen = atomicCAS(mem + x, old, new);  
} while (old != seen) —————→ this is called “spinning”
```
- can be used to build spinning locks



Atomic Operations

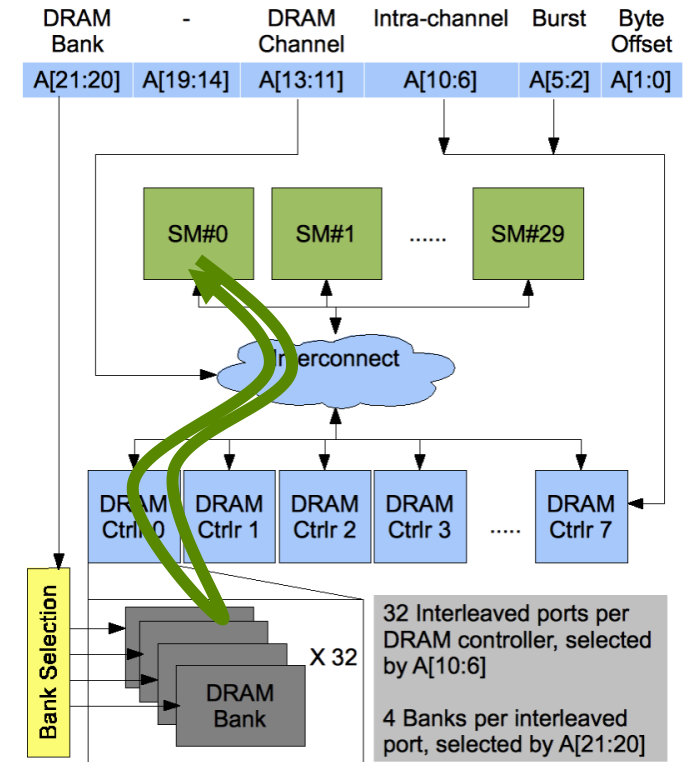
□ Parallel histogram in CUDA with atomic operations

```
__global__ void hist_kernel(unsigned char *buffer, long size,
                           unsigned int *hist)
{
    int tid = threadIdx.x + blockIdx.x * blockDim.x;
    int stride = blockDim.x * gridDim.x;

    for (unsigned int i = tid; i < size; i += stride ) {
        int alphabet_position = buffer[i] - 'a';
        if (alphabet_position >= 0 && alphabet_position < 26)
            atomicAdd(&(hist[alphabet_position/4]), 1);
    }
}
```

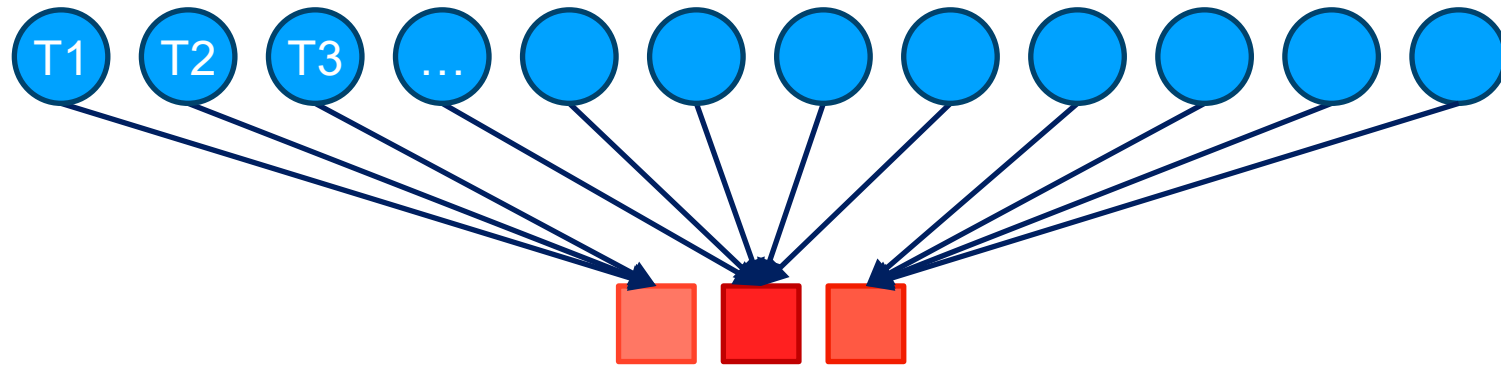
Atomic Operations

- ❑ Performance considerations:
 - Atomic operations impact latency and throughput: they partially serialize execution
 - Even worse, they serialize the most time consuming part of a kernel, that is memory accesses
 - Latency of atomic operations: memory load latency + memory store latency
- ❑ Atomic operations can access different types of memory (global, shared, caches), with performance varying accordingly



Atomic Operations – Worst-case

- ❑ Worst-case scenario: many threads attempt to do an atomic operation on the same location, execution becomes serialized, only one thread can update that location at a time
 - Throughput limit = rate at which the application can execute an atomic operation on a contended memory location
 - The latency of the sequence of atomic operations will likely dominate the kernel's execution time



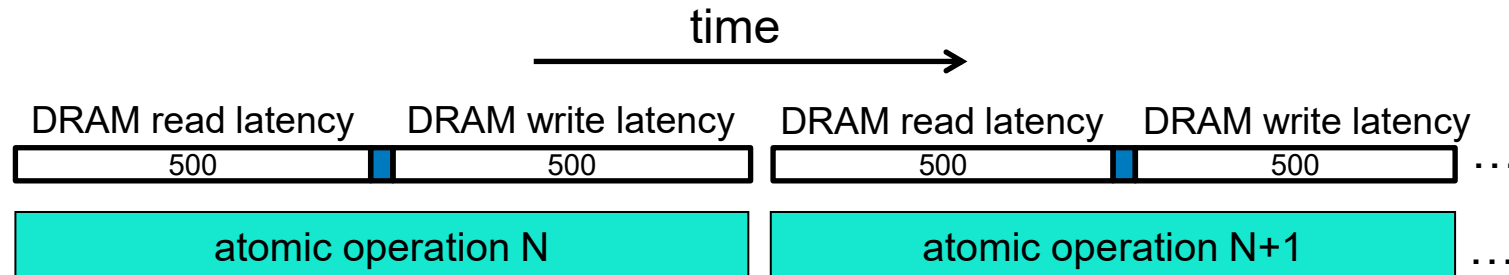
Atomic Operations – Worst-case

- ❑ The total latency for an atomic operation on global memory (DRAM) is in the order of 1000 cycles (500 read, 500 write)
- ❑ Assume a 64-bit DDR DRAM interface with 8 channels @ 1 GHz, its peak access throughput is

$$8 \left(\frac{\text{bytes}}{\text{transfer}} \right) \cdot 2 \left(\frac{\text{transfers}}{\text{clock} \cdot \text{channel}} \right) \cdot 1 \text{ G} \left(\frac{\text{clocks}}{\text{second}} \right) \cdot 8 (\text{channels}) = 128 \text{ GB/s}.$$

- ❑ The worst-case throughput is 1 atomic operation every 1000 cycles. This gives a time-based throughput for 32-bit atomics of only

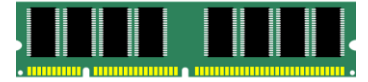
$$\frac{1}{1000} \left(\frac{\text{atomics}}{\text{clock}} \right) \cdot 1 \text{ G} \left(\frac{\text{clocks}}{\text{second}} \right) = 2.5 \text{ M} \left(\frac{\text{atomics}}{\text{second}} \right) \text{ or } 10 \text{ MB/s} \quad \text{OUCH!}$$



Atomic Operations

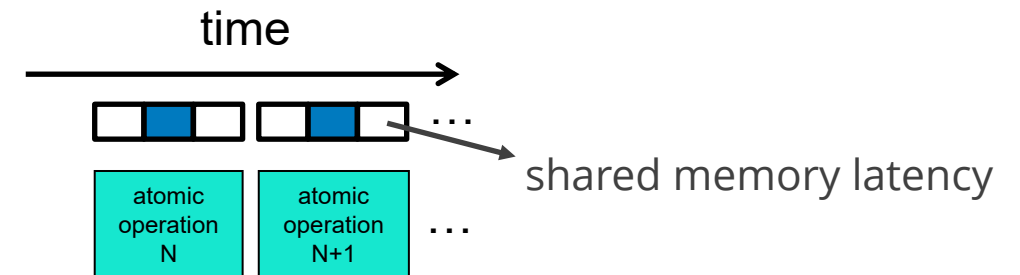
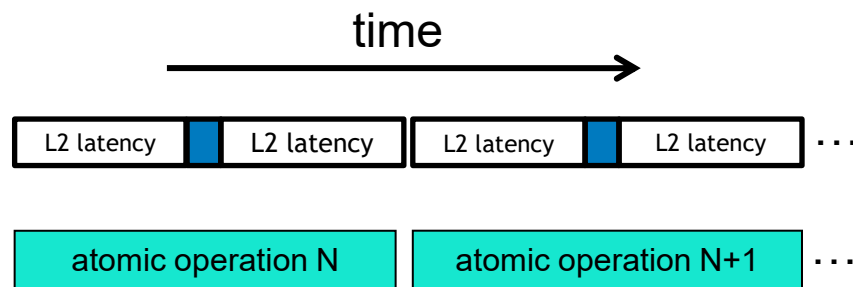


We need to do better than hitting DRAM with atomics...



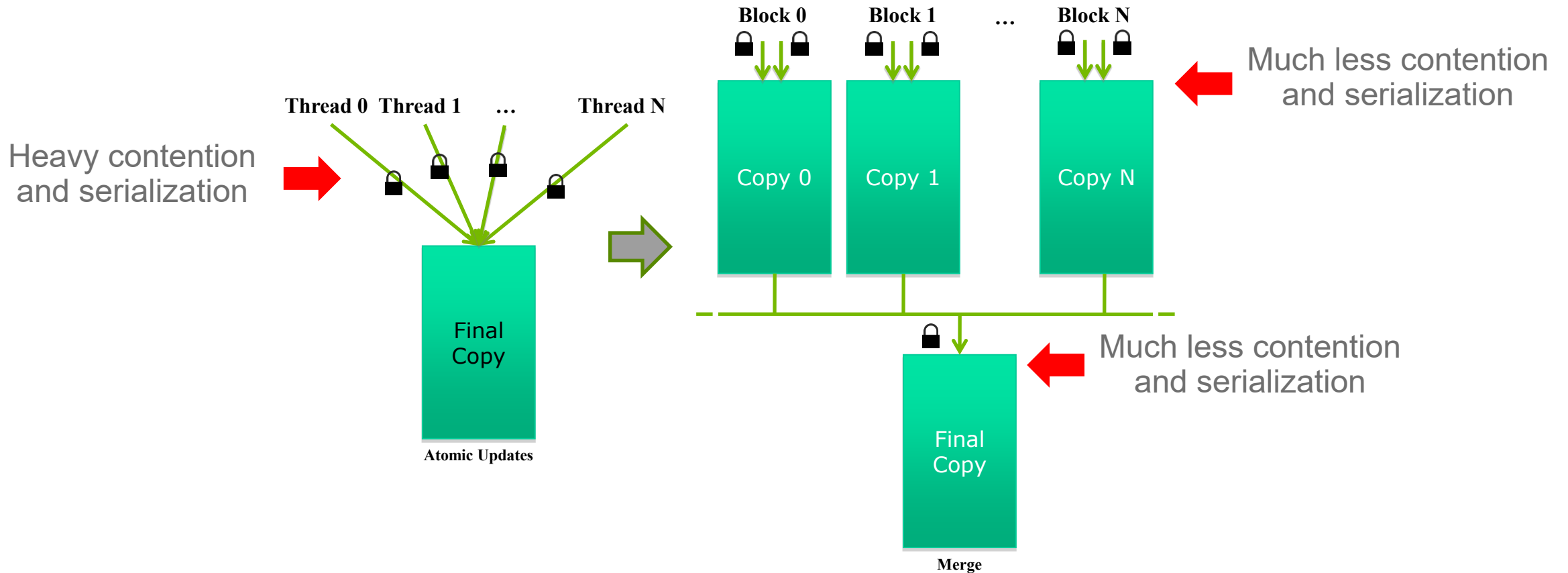
maybe not...

- ❑ Atomic operations on L2 cache
 - About 1/10 of the DRAM latency
 - Shared among all blocks
- ❑ Atomic operations on shared memory
 - Very short latency
 - Private to each thread block



Privatization

- ❑ Replicate contended data structures into private copies that each subset of threads can independently
- ❑ Each copy will later be merged into the original



Privatization

- ❑ Privatization is a powerful and frequently used technique for parallelizing output-centric applications
- ❑ Costs:
 - × Overhead for creating and initializing private copies
 - × Overhead for accumulating the contents of private copies into the final shared copy
- ❑ Benefits:
 - ✓ Much less contention and serialization in accessing both the private copies and the final copy
 - ✓ Private copies can be held in the faster shared memory, already improving performance by 10x
- ❑ Don't overuse it: carefully balance between reducing the amount of contention and the merging cost

Privatization

- ❑ Privatized variables must be updated only by **associative** and **commutative** operations
 - Histogram's increment operation is associative and commutative
- ❑ The private data shall fit into shared memory
 - Works for small histograms
 - It is possible to partially privatize the histogram and only go to the copy in global memory when needed

Hence, the histogram kernel is a great candidate for privatization:

- All threads working on an input section are in the same block
- Threads in the same block can access shared memory
- Throughput for atomics on shared memory is 10-100x better than on DRAM or L2 cache

Parallel Histogram in CUDA with Privatization

```
__global__ void hist_privatized_kernel(unsigned char* buffer, unsigned int* hist,
unsigned int num_elements, unsigned int num_bins) {
    unsigned int tid = blockIdx.x * blockDim.x + threadIdx.x;
    extern __shared__ unsigned int hist_s[]; // private copy

    for(unsigned int binIdx = threadIdx.x; binIdx < num_bins; binIdx += blockDim.x)
        hist_s[binIdx] = 0u;

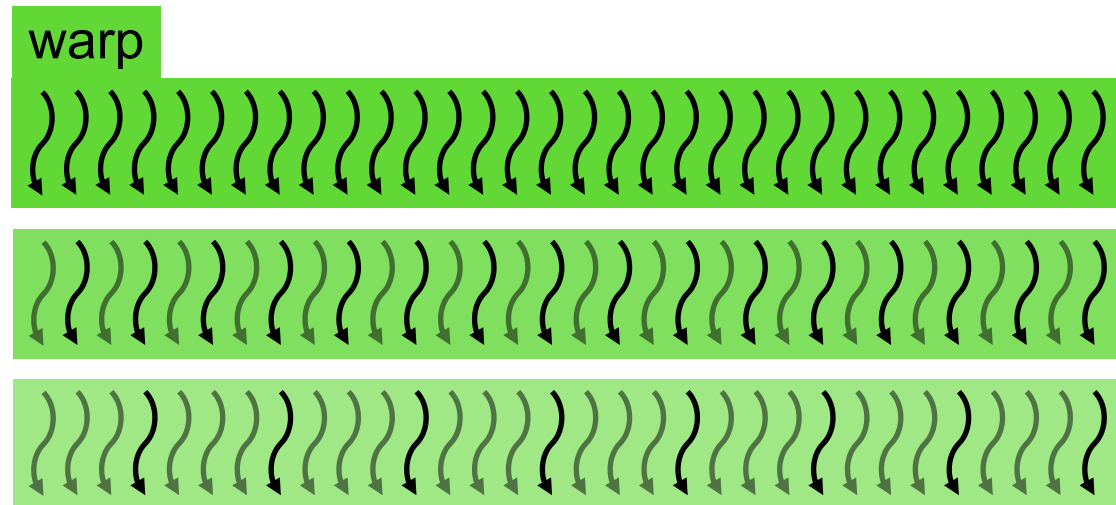
    __syncthreads();

    for (unsigned int i = tid; i < num_elements; i += blockDim.x * gridDim.x) {
        int alphabet_position = buffer[i] - 'a';
        if (alphabet_position >= 0 && alphabet_position < 26)
            atomicAdd(&(hist_s[alphabet_position/4]), 1);
    }
    __syncthreads();

    for(unsigned int binIdx = threadIdx.x; binIdx < num_bins; binIdx += blockDim.x)
        atomicAdd(&(hist[binIdx]), hist_s[binIdx]); // merge into the original
}
```

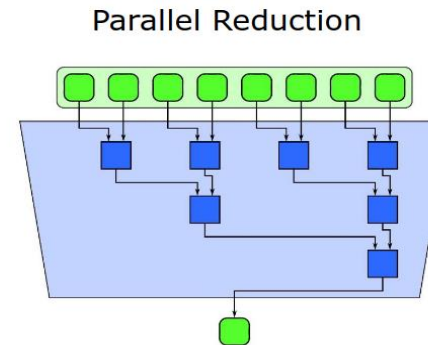
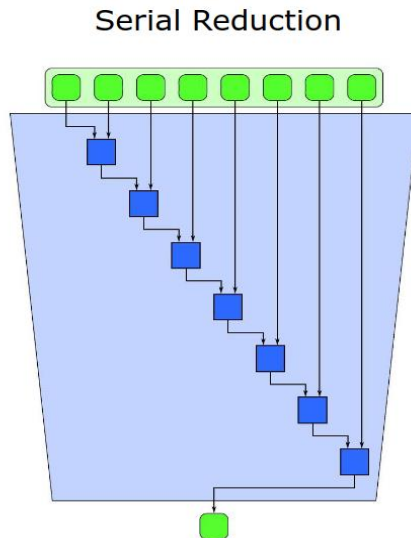
Reduction

- ❑ The reduction (e.g. array elements sum) is another typical computation pattern that can be parallelized
- ❑ We can use it to analyze
 - Work efficiency
 - Resource efficiency
- ❑ Key issue to solve: **control divergence**



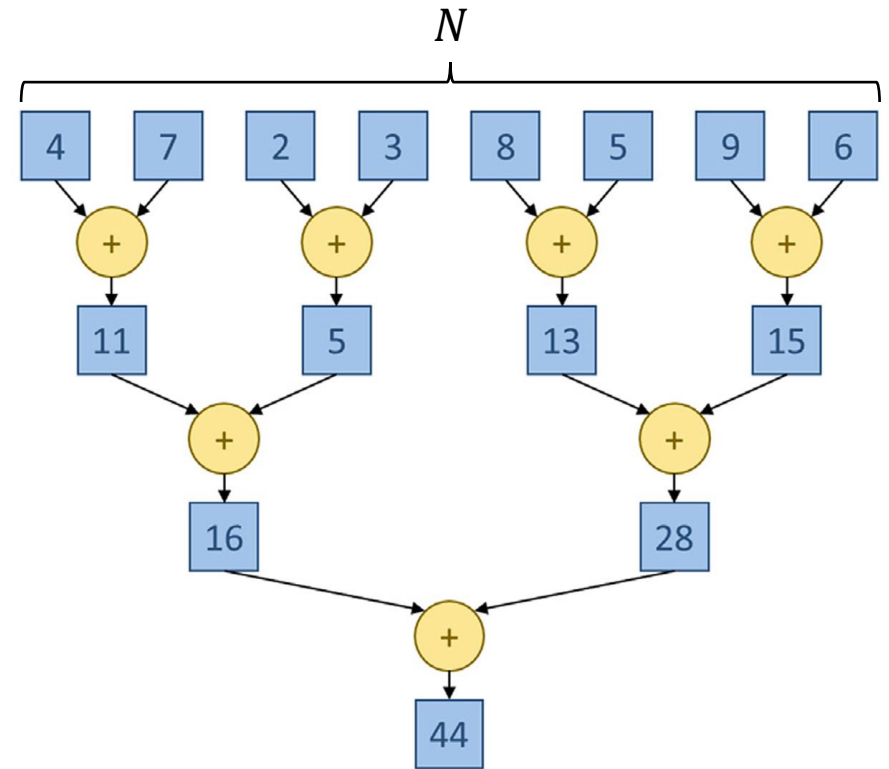
Reduction

- ❑ Typically a part of other, more complex, kernels
- ❑ Uses an associative (and commutative) function/operator (sum, max, logical, ...) to combine all elements of a collection into a single result
- ❑ From ex. session 1, it has:
 - sequential complexity $O(n)$
 - Parallel complexity $O(\log n)$



Reduction

- Parallel binary reduction tree
 - Assuming there are enough resources, it can perform $N - 1$ operations in $\log_2(N)$ rounds
 - Average parallelism (number of concurrent operations): $\frac{N-1}{\log_2 N}$
- This is work efficient, but not resource efficient
 - We use $N/2$ processors, but more and more remain inactive, as seen from $\frac{N}{2} > \frac{N-1}{\log_2 N}$



Reduction

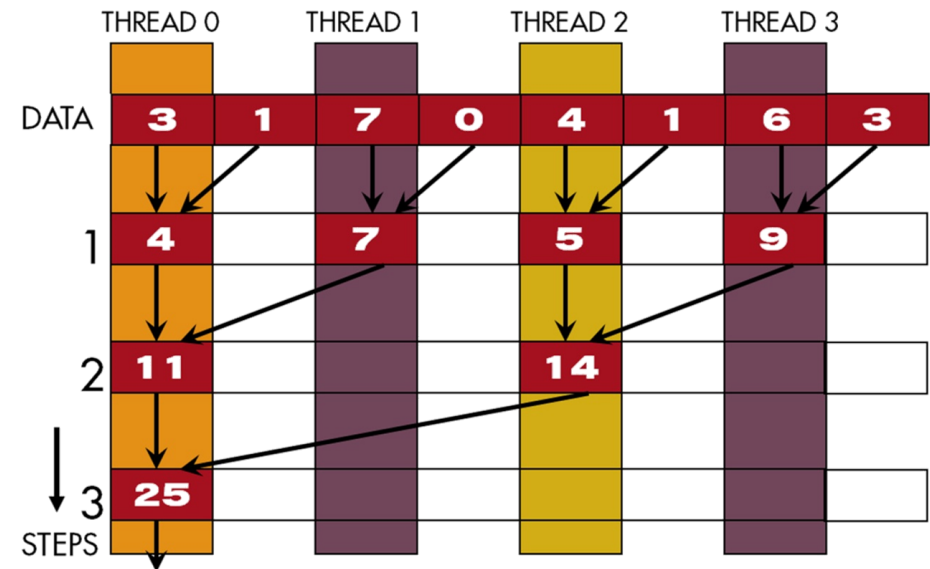
- ❑ Simple parallel implementation

- $N/2$ threads
- $\log_2(N)$ steps
- At each step each thread operates on 2 elements, then the number of threads is halved

- ❑ In-place reduction in shared memory

- Reduced global memory traffic

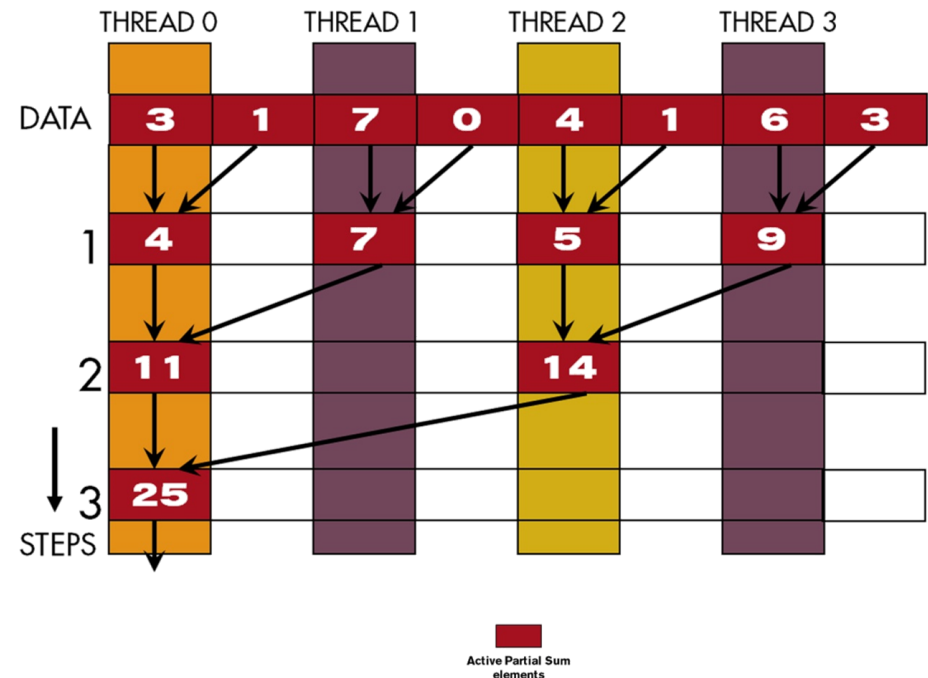
- ❑ N has to be lower than the maximum number of threads in a block



Reduction

- ❑ Thread to data mapping:
 - Each thread writes to an even-index location in the partial sum
 - One of the inputs comes from the same index
 - The second input is increasingly farther away

- ❑ For now, let us assume that a single block exists and see what happens inside it



Parallel Reduction

□ Baseline CUDA implementation (single block)

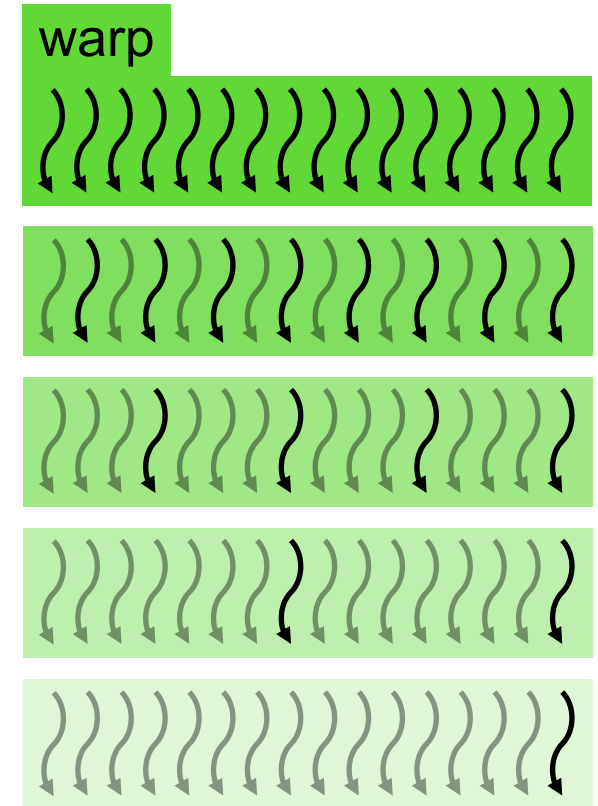
```
__global__ void simple_sum_reduction(float* input, float* output) {
    __shared__ float partial_s[(SIZE+1)/2]; // assume blockDim.x == SIZE/2
    unsigned int t = threadIdx.x;
    partial_s[t] = input[t] + input[t + blockDim.x];

    for (unsigned int stride = 1; stride <= blockDim.x; stride *= 2) {
        __syncthreads();
        if (t % stride == 0) {
            partial_s[t] += partial_s[t + stride];
        }
    }

    if (threadIdx.x == 0) {
        *output = partial_s[0];
    }
}
```

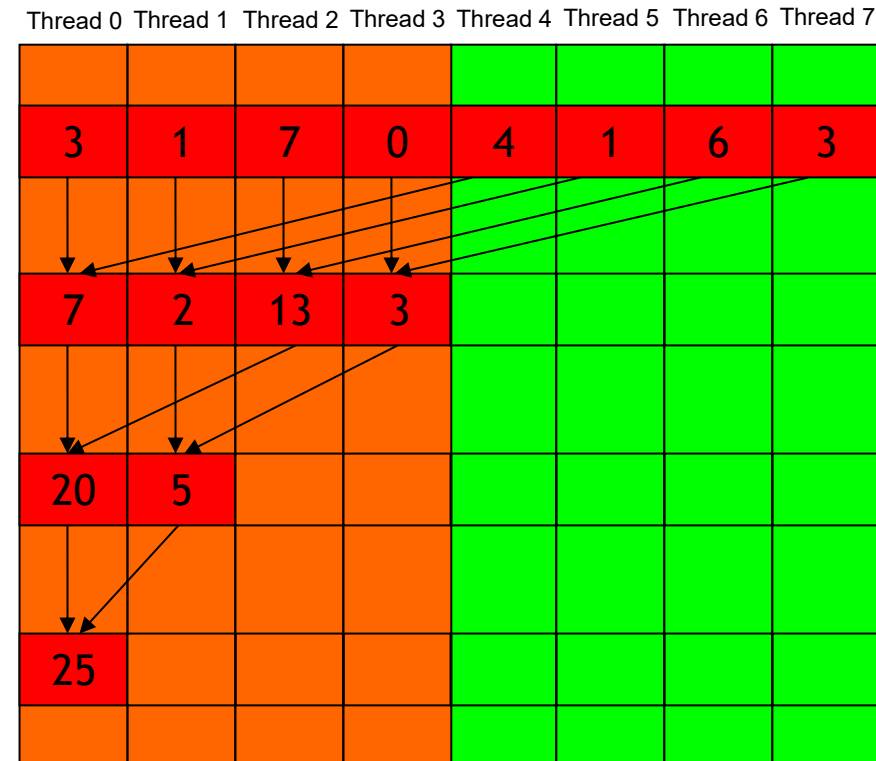
Suboptimal Implementation

- ❑ At each iteration, threads end up following two distinct control flow paths:
 - Some threads still perform additions, others do not
 - Threads that do not perform addition still exist and take up hardware resources
- ❑ One out of every contiguous pairs of active threads goes idle the next iteration
 - **Warps will always diverge!**
 - Half (or less) of the threads will be executing after the first iteration, in an alternating pattern
 - Only after the 5th iteration, entire warps will be idle and thus not diverge (but still waste resources)



Optimized Implementation

- ❑ Change index usage to reduce divergence
 - Possible because reduction operators are commutative and associative
- ❑ The upper half of the active threads goes idle after each iteration
- ❑ Partial sums are stored in the front of the array



Optimized Parallel Reduction

❑ Improved CUDA implementation (single block)

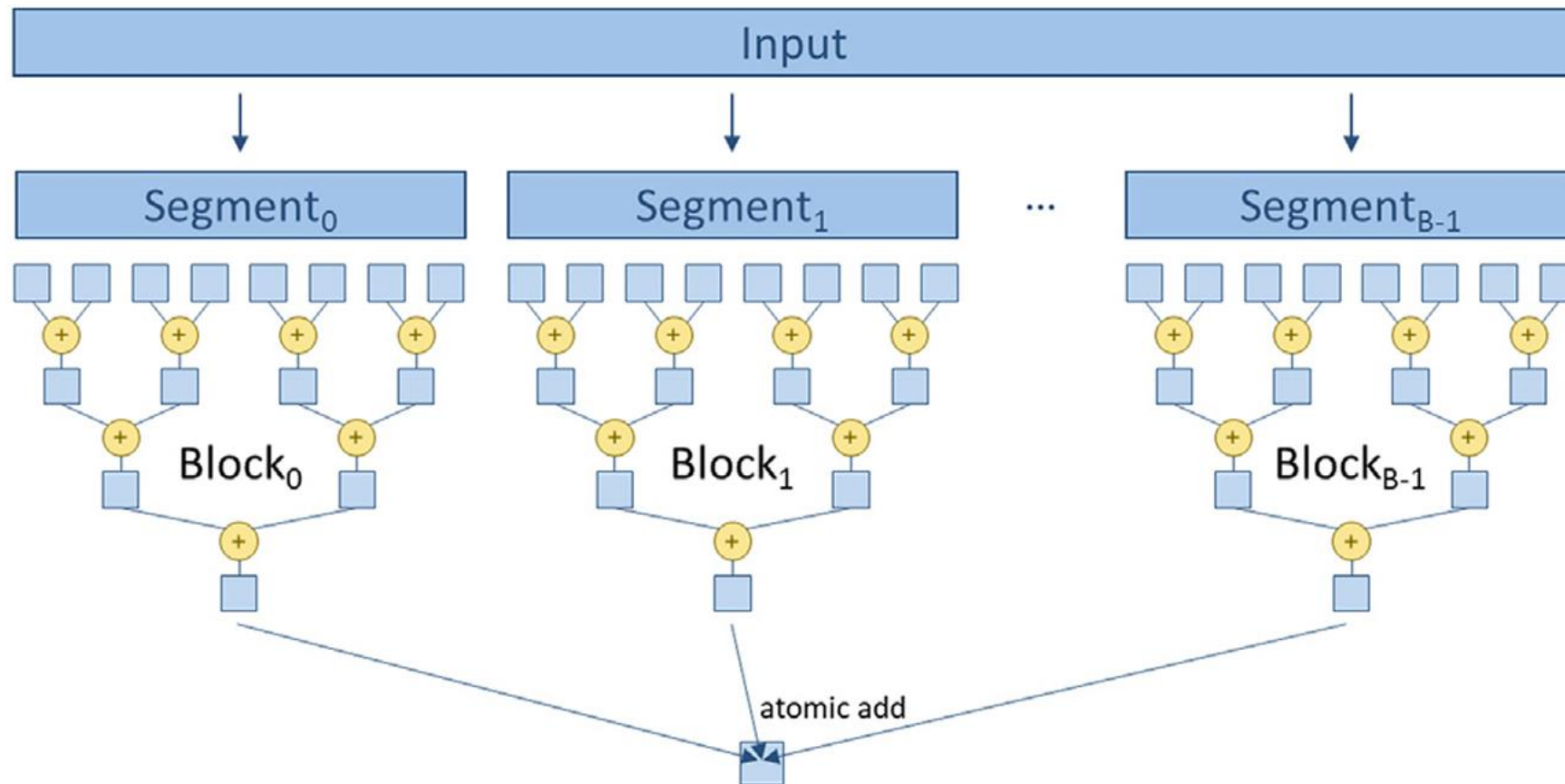
```
__global__ void simple_sum_reduction(float* input, float* output) {
    __shared__ float partial_s[(SIZE+1)/2]; // assume blockDim.x == SIZE/2
    unsigned int t = threadIdx.x;
    partial_s[t] = input[t] + input[t + blockDim.x];

    for (unsigned int stride = blockDim.x/2; stride >= 1; stride /= 2) {
        __syncthreads();
        if (t < stride) {
            partial_s[t] += partial_s[t + stride];
        }
    }

    if(threadIdx.x == 0) {
        *output = partial_s[0];
    }
}
```

Arbitrary Input Length, Multiple Blocks

- ❑ Reduce as seen inside each block, using shared memory
- ❑ **Atomically** update a common final output once each block finishes (done by the last active thread of the block)



Hierarchical Parallel Reduction

□ Final CUDA implementation (multiple blocks)

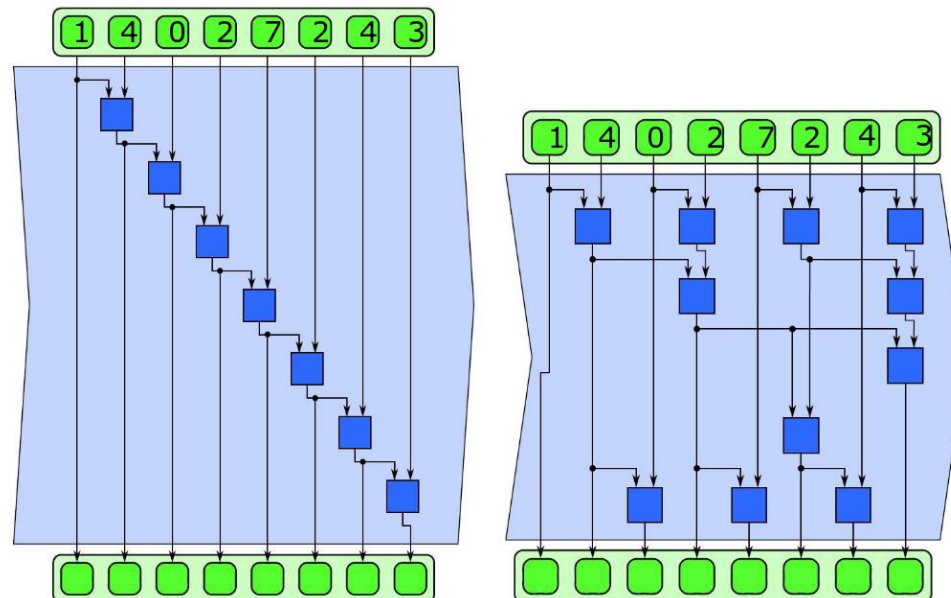
```
__global__ void full_sum_reduction(float* input, float* output) {
    __shared__ float partial_s[(SIZE+1)/2]; // assume blockDim.x == SIZE/2
    unsigned int g = blockIdx.x * blockDim.x * 2 + threadIdx.x;
    unsigned int t = threadIdx.x;
    partial_s[g] = input[g] + input[g + blockDim.x];

    for (unsigned int stride = blockDim.x/2; stride >= 1; stride /= 2) {
        __syncthreads();
        if (t < stride) {
            partial_s[t] += partial_s[t + stride];
        }
    }

    if (threadIdx.x == 0) {
        atomicAdd(output, partial_s[0]);
    }
}
```

Scan (Prefix Sum)

- ❑ Computing prefix sums requires computing and returning all partial results of a sequential reduction
- ❑ The sequential scan is work efficient, parallel scan can be slower in some cases
- ❑ Key issue to solve: **work efficiency**



Scan (Prefix Sum)

```
y[0] = x[0];  
for (i = 1; i < Max_i; i++)  
    y[i] = y[i-1] + x[i];
```

- ❑ Sequential C implementation of an inclusive (i-th value included in the i-th result) prefix sum
 - N additions for N elements, $O(N)$ complexity
 - Only one operation more than sequential reduction
 - Computationally efficient!

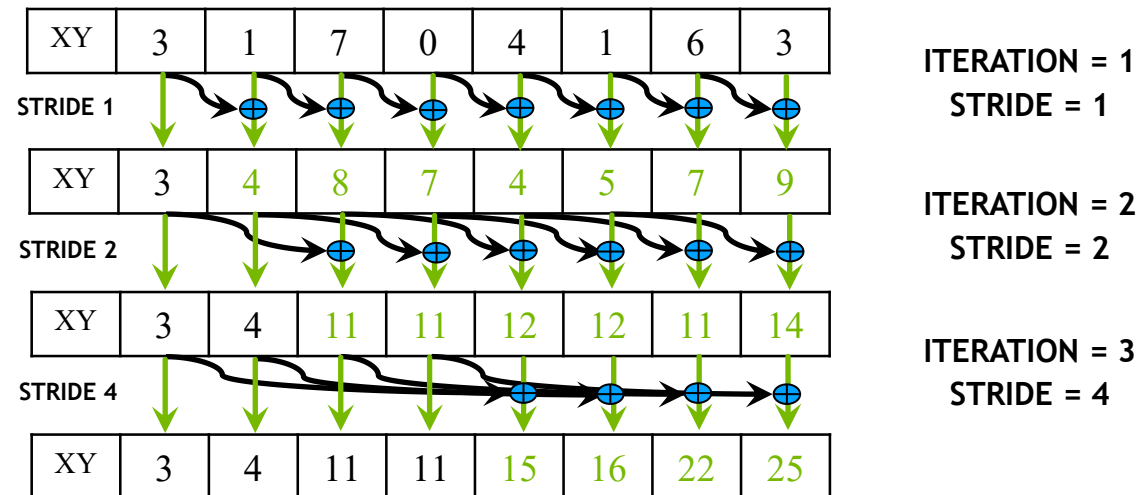
Scan (Prefix Sum)

$$\begin{aligned} y_0 &= x_0 \\ y_1 &= x_0 + x_1 \\ y_2 &= x_0 + x_1 + x_2 \\ y_3 &= x_0 + x_1 + x_2 + x_3 \\ &\dots \end{aligned}$$

- ❑ Naive parallel implementation of an inclusive addition scan
 - Threads work independently on each prefix
 - **A lot of redundant computation!**
 - The last thread is performing a sequential reduction, so complexity is still $O(N)$

Scan (Prefix Sum)

- Parallel implementation with interleaved reduction trees
 - Read input into shared memory
 - Thread j adds elements j and $j - \text{stride}$ and writes result into element j
 - Double the stride and repeat



Simple Parallel Scan

❑ Work-inefficient CUDA implementation (single block)

```
__global__ void poor_prefix_sum(float* input, float* output) {
    __shared__ float prefix_s[SIZE];
    unsigned int t = threadIdx.x;

    prefix_s[t] = input[t];
    __syncthreads();
    for (unsigned int stride = 1; stride < blockDim.x; stride *= 2) {
        float val = 0.0f;
        if (t >= stride)
            val = prefix_s[t - stride];
        __syncthreads();
        prefix_s[t] += val;
        __syncthreads();
    }

    output[g] = prefix_s[t];
}
```

Work Efficiency

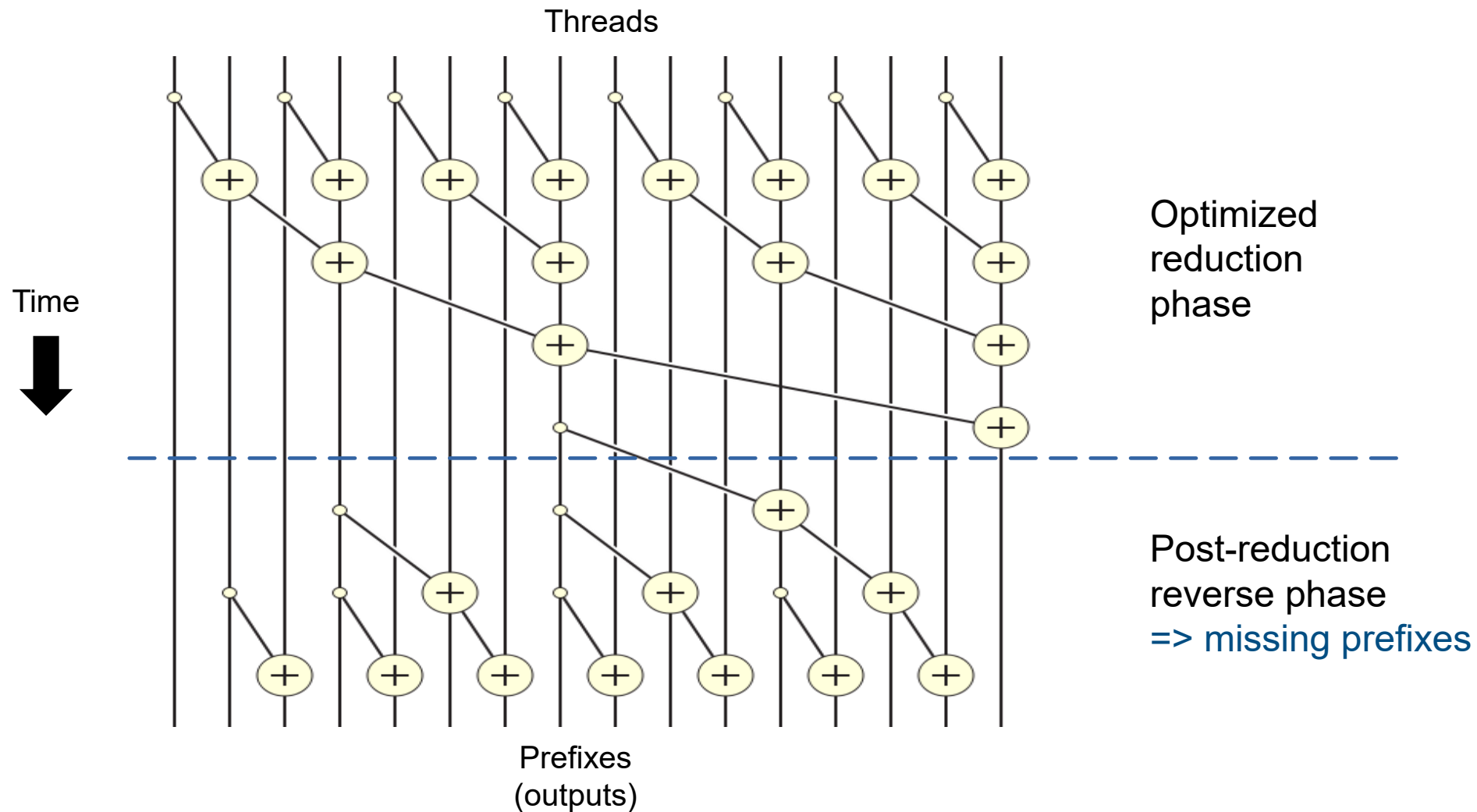
- ❑ The parallel scan executes $\log_2(N)$ iterations
- ❑ The iterations do $(N - 1), (N - 2), (N - 4), \dots (N - N/2)$ additions each
 - Total number of additions: $O(N \cdot \log_2(N))$
- ❑ Compared to $O(N)$ for the sequential implementation
 - 10x overhead with $N = 1024$
- ❑ Work inefficient and leaves behind a lot of inactive hardware resources

Def: work efficiency means the total amount of computation performed by the parallel algorithm is asymptotically the same as that of the optimal sequential algorithm

Optimized Parallel Scan

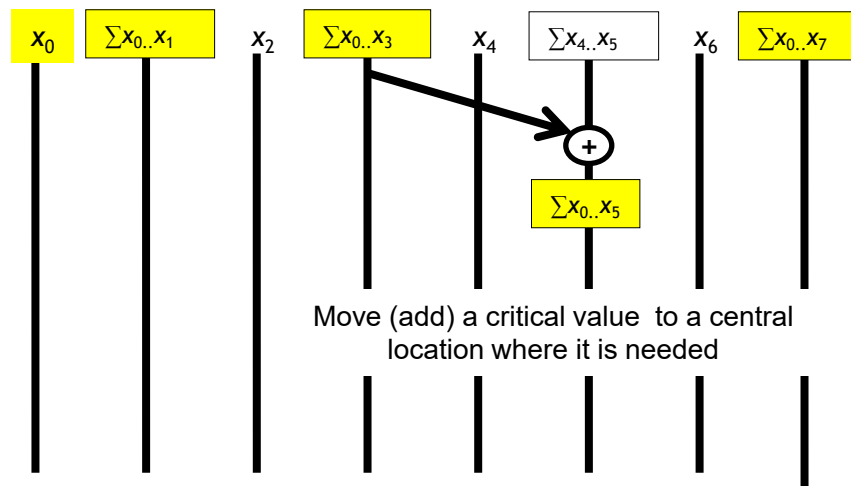
- ❑ Improving efficiency with:
 - Two-phased approach
 - Reuse of intermediate results
 - Improved thread-index to data-index mapping
- ❑ The two-phased approach resembles two reduction trees, with the second one upside-down
 - Perform an optimized reduction (leaves to root)
 - Build missing output values (root to leaves)
- ❑ For ease of explanation, assume again that the whole computation fits in one block

Optimized Parallel Scan

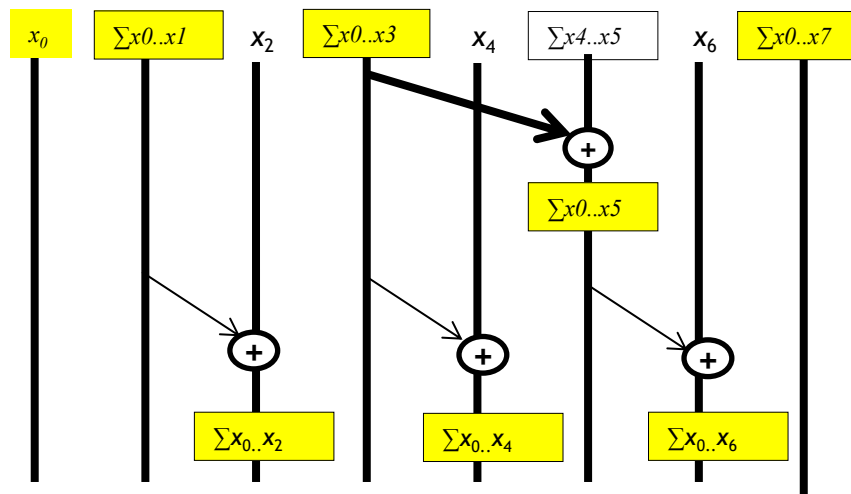


Optimized Parallel Scan

Same concept, yet another drawing!



Post-reduction
reverse phase



Optimized Parallel Scan

□ Reduction phase (single block)

```
__global__ void prefix_sum(float* input, float* output) {
    __shared__ float prefix_s[SIZE]; // assume blockDim.x == SIZE/2
    unsigned int t = threadIdx.x;

    prefix_s[t] = input[t];
    prefix_s[t + blockDim.x] = input[t + blockDim.x];
    __syncthreads();

    for (unsigned int stride = 1; stride < SIZE; stride *= 2) {
        __syncthreads();
        unsigned int index = (t + 1) * stride * 2 - 1;
        if (index < SIZE)
            prefix_s[index] += prefix_s[index - stride];
    }

    // ...
}
```


Optimized Parallel Scan

❑ Reverse phase (finish up missing prefixes) (single block)

```
// ...
if (t == 0)
    prefix_s[SIZE - 1] = 0.0f;
__syncthreads();

for (unsigned int stride = SIZE; stride >= 1; stride /= 2) {
    __syncthreads();
    unsigned int index = (t + 1) * stride * 2 - 1;
    if (index + stride < SIZE) { // transforms the tree into prefix sums
        float temp = prefix_s[index];
        prefix_s[index] = prefix_s[index + stride];
        prefix_s[index + stride] += temp;
    }
}
__syncthreads();

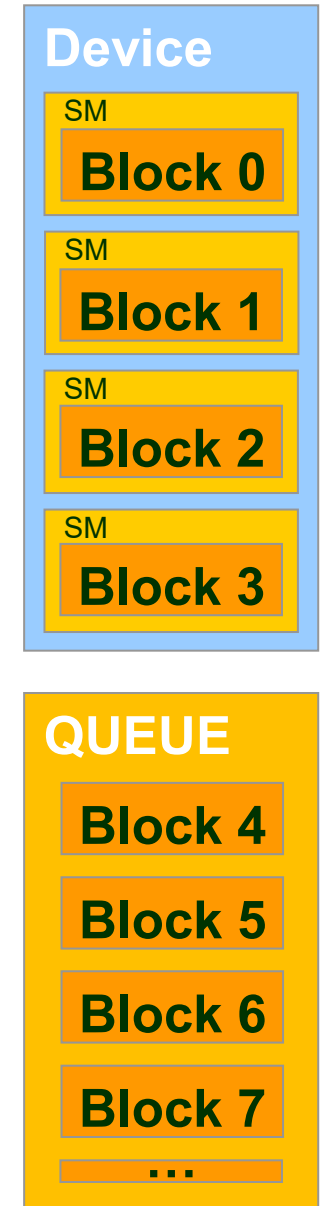
output[t] = prefix_s[t] + input[t]; // convert to inclusive scan
output[t + blockDim.x] = prefix_s[t + blockDim.x] + input[t + blockDim.x];
}
```

Optimized Parallel Scan

- ❑ The work efficient version executes $\log_2(N)$ iterations in the reduction step
 - Total number of operations: $N - 1$
- ❑ In the reverse phase the iterations do $(2 - 1)$, $(4 - 1)$, $(N/2 - 1)$ operations
 - Total: $(N - 2) - (\log_2(N) - 1)$
- ❑ Both phases are $O(N)$ and they do not perform more than $2(N - 1)$ operations
- ❑ With enough hardware resources, the 2x extra work is compensated by the reduced execution time

Thread Coarsening

- ❑ Assume now to have an extremely large input size, so much so that there aren't enough streaming multiprocessors to run simultaneously all the thread blocks launched for it
- ❑ The hardware will serialize the surplus thread blocks, executing a new thread block whenever an old one has completed
- ❑ But this means that:
 - Any thread going idle frees up resources that don't get used by a waiting block until the thread's whole block finishes executing ⇒ underutilization
 - We waste time setting up parallel threads just for them to run in a serial-like manner

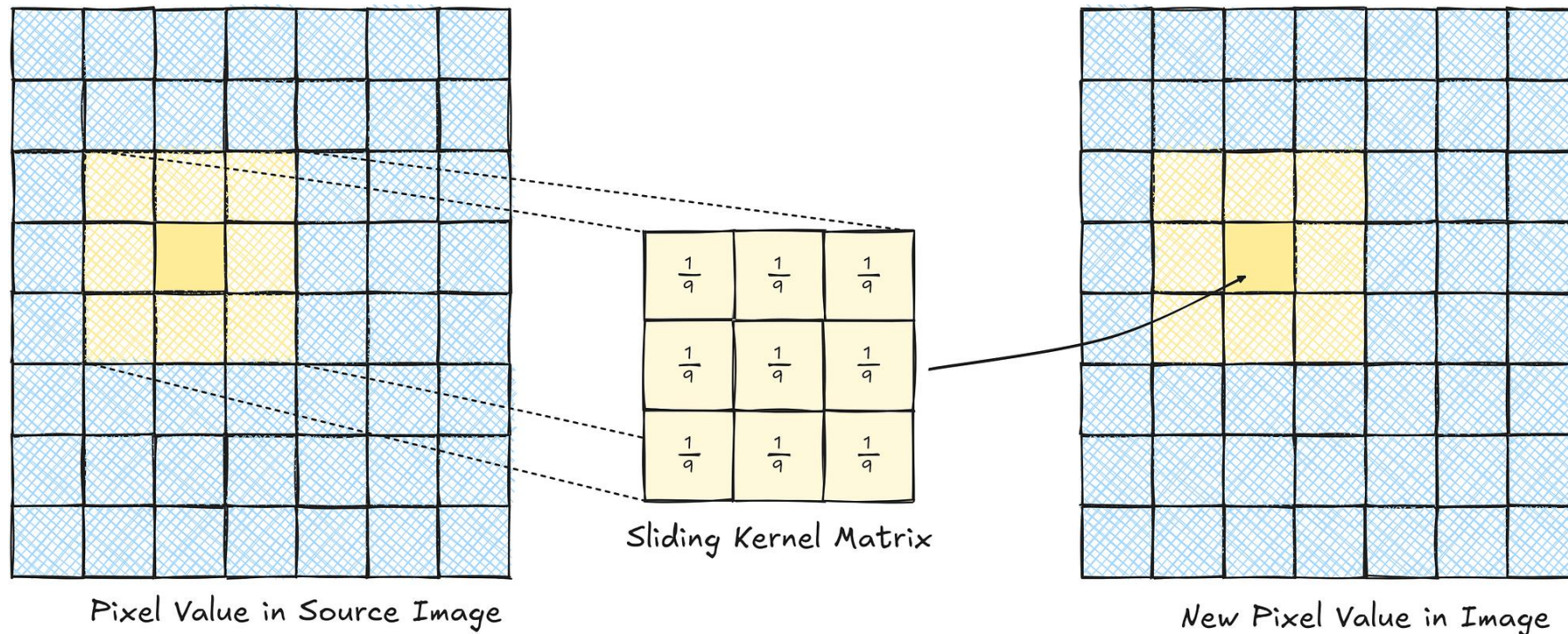


Thread Coarsening

- ❑ Parallelizing work might have a **price**, e.g.:
 - redundant loading of data by different blocks
 - redundant work
 - synchronization overhead
- ❑ When this is the case, finer parallelism granularities (more threads) come with the highest price
- ❑ When threads are executed in parallel by the hardware, this price of parallelism is worth paying
- ❑ But, if the work gets serialized because of insufficient hardware resources, this price is unnecessary
 - In this case, it is better for the programmer to **manually serialize part of the work** inside threads and reduce the price that is paid for parallelism

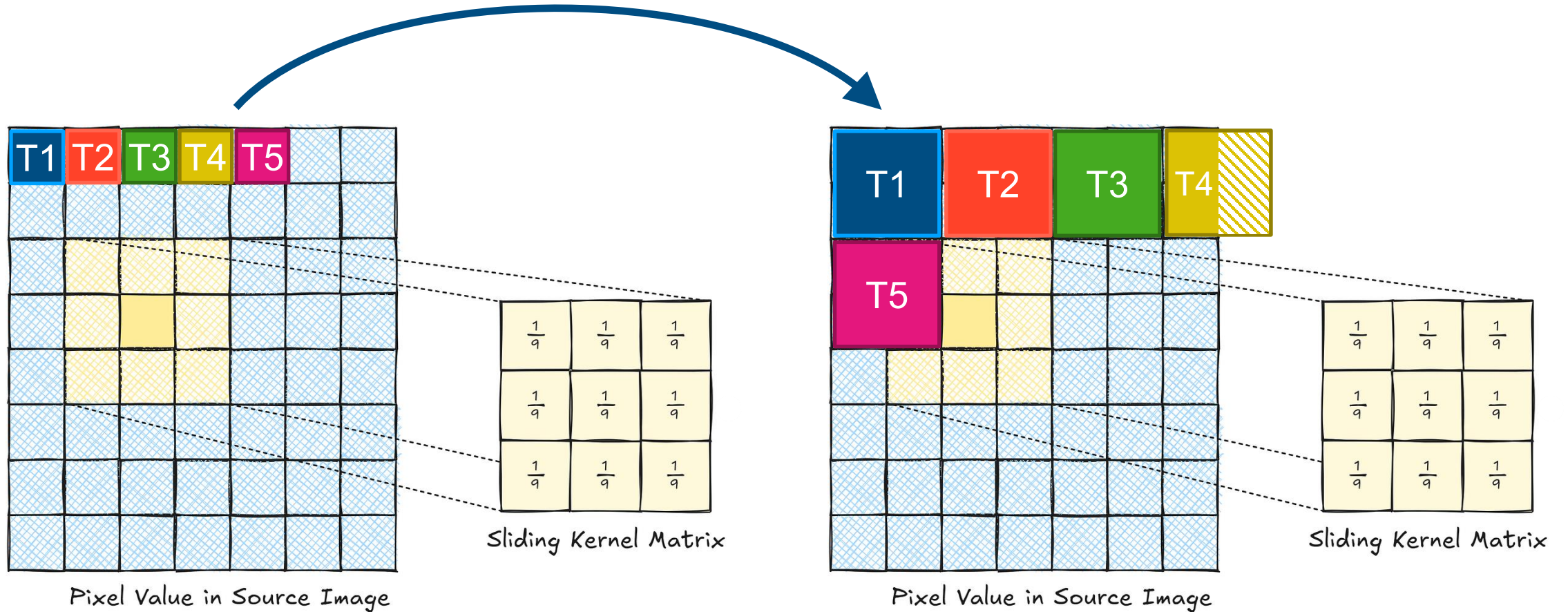
Thread Coarsening

- ❑ How: by assigning to each thread multiple units of work
- ❑ We already did this for the histogram, by giving portions of the input per-thread, instead of single values
- ❑ Let us see how to implement it for a simple blur kernel (task: update each pixel as the average of its neighbors)



Thread Coarsening

- How: by assigning to each thread multiple units of work



Thread Coarsening

- ❑ How to coarsen in practice:
 - Wrap the per-thread independent part of your kernel-code with a coarsening loop
 - Each coarsening iteration runs an entirely unrelated part of the kernel using the same thread
 - No calls to `__syncthreads()` should be made in the new coarsening loop because the threads act independently
 - Usually this means the coarsening loop engulfs either the initial part of the kernel, up to its main loop, or the kernel as a whole

Basic Blur Kernel

```
__global__ void blur_kernel(double* in, double* out, int w, int h) {
    int blurCol = threadIdx.x + blockIdx.x * blockDim.x;
    int blurRow = threadIdx.y + blockIdx.y * blockDim.y;

    for(int blurRow = -BLUR_SIZE; blurRow < BLUR_SIZE+1; ++blurRow) {
        for(int blurCol = -BLUR_SIZE; blurCol < BLUR_SIZE+1; ++blurCol) {
            int curRow = Row + blurRow;
            int curCol = Col + blurCol;
            if(curRow > -1 && curRow < h && curCol > -1 && curCol < w) {
                pixVal += in[curRow * w + curCol];
                pixels++;
            }
        }
    }
    out[Row * w + Col] = pixVal / pixels;
}
```


Coarsened Blur Kernel

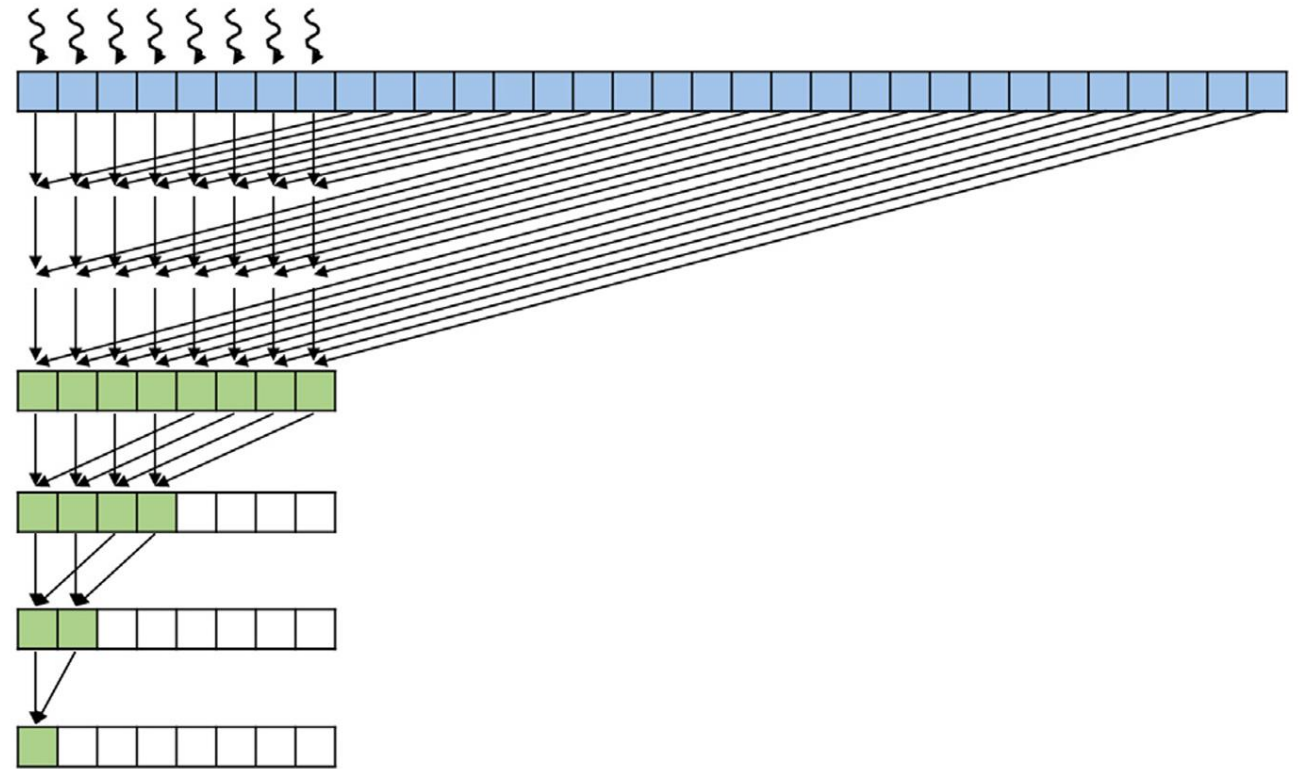
```
__global__ void blur_kernel_coarsened(double* in, double* out, int w, int h) {
    int baseCol = (blockIdx.x * blockDim.x + threadIdx.x) * COARSEN_FACTOR_X;
    int baseRow = (blockIdx.y * blockDim.y + threadIdx.y) * COARSEN_FACTOR_Y;
    for (int dy = 0; dy < COARSEN_FACTOR_Y; ++dy) {
        int Row = baseRow + dy;
        if (Row >= h) continue;
        for (int dx = 0; dx < COARSEN_FACTOR_X; ++dx) {
            int Col = baseCol + dx;
            if (Col >= w) continue;
            double pixVal = 0.0;
            int pixels = 0;
            for (int blurRow = -BLUR_SIZE; blurRow <= BLUR_SIZE; ++blurRow) {
                for (int blurCol = -BLUR_SIZE; blurCol <= BLUR_SIZE; ++blurCol) {
                    int curRow = Row + blurRow;
                    int curCol = Col + blurCol;
                    if (curRow >= 0 && curRow < h && curCol >= 0 && curCol < w) {
                        pixVal += in[curRow * w + curCol];
                        pixels++;
                    }
                }
            }
            out[Row * w + Col] = pixVal / pixels;
        }
    }
}
```

Thread Coarsening

- ❑ Coarsening has **drawbacks**, must be used with care!
 - Use it only when necessary (iff there is a non-negligible price to pay for each parallel instance)
 - Don't use too much, otherwise hardware becomes underutilized, always expose enough parallelism to achieve transparent scalability (the hardware is still free to dynamically allocate some work)
 - Don't bloat threads too much, otherwise they could exceed a SM's limited resources and hurt occupancy

Coarsened Reduction

- ❑ To parallelize reduction, we distributed the work across multiple thread blocks
- ❑ Price paid: as threads become inactive, each block sees more idle and diverged warps...
 - Hardware gets underutilized
- ❑ Fix: let each block handle multiple input sections!
- ❑ Now loads and compute of different sections can overlap!
- ❑ Most useful when input size is veeeeeeery large!

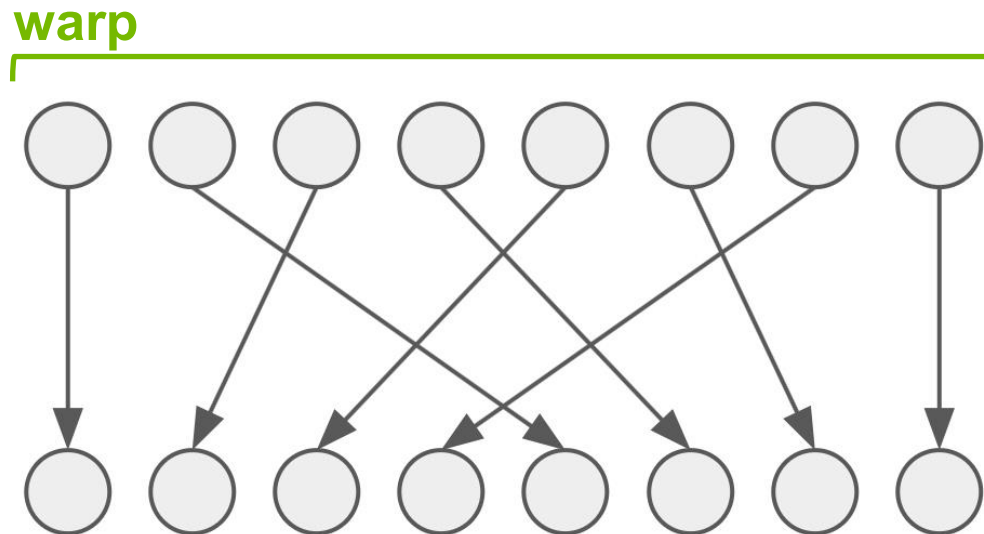


Optimizations Recap

Optimization	Benefit to compute cores	Benefit to memory	Strategies
Maximizing occupancy	More work to hide pipeline latency	More parallel memory accesses to hide DRAM latency	Tuning usage of SM resources such as threads per block, shared memory per block, and registers per thread.
Enabling coalesced global memory accesses	Fewer pipeline stalls waiting for global memory accesses	Less global memory traffic and better utilization of bursts / cache lines	Transfer between global memory and shared memory in a coalesced manner and performing uncoalesced accesses in shared memory. Rearranging the mapping of threads to data. Rearranging the layout of the data.
Minimizing control divergence	High SIMD efficiency (fewer idle cores during SIMD execution)	--	Rearranging the mapping of threads to work and/or data. Rearranging the layout of the data.
Tiling of reused data	Fewer pipeline stalls waiting for global memory accesses	Less global memory traffic	Placing data that is reused within a block in shared memory or registers so that it is transferred between global memory and the SM only once.
Privatization	Fewer pipeline stalls waiting for atomic updates	Less contention and serialization of atomic updates	Applying partial updates to a private copy of the data and then updating the universal copy when done.
Thread coarsening	Less redundant work, divergence, or synchronization	Less redundant global memory traffic	Assigning multiple units of parallelism to each thread to reduce the price of parallelism when it is incurred unnecessarily.

Extra: Warp Shuffles

- ❑ While the high performance obtained by warp execution happens behind the scene, many CUDA programs can achieve even higher performance by using explicit warp-level programming
- ❑ In short: warp shuffles are a faster mechanism for moving data between threads in the same warp



Extra: Warp Shuffles

❑ There are 4 main primitives:

- `T __shfl_up_sync(unsigned mask, T var, unsigned int delta):`
copy `var` from a lane with `ID = caller_ID - delta` to the caller
- `T __shfl_down_sync(unsigned mask, T var, unsigned int delta):`
copy `var` from a lane with `ID = caller_ID + delta` to the caller
- `T __shfl_xor_sync(unsigned mask, T var, unsigned int laneMask):`
copy `var` from a lane with `ID = caller_ID ^ laneMask` to the caller
- `T __shfl_sync(unsigned mask, T var, unsigned int srcLane):`
copy `var` from the lane with `ID = srcLane` to the caller

- ❑ The mask decides which threads partake in the shuffle (usually `-1` aka everyone)
- ❑ These are “syncs”, they are executed as soon as all participating threads reach them
- ❑ The generic type `T` can be unsigned int, long long, unsigned long long, float, double
- ❑ If you want to sync a warp and do nothing else, use `__syncwarp(unsigned mask)`

Extra: Warp Shuffles

- ❑ Can be viewed as a “synchronized data exchange” directly to and from registers, the fastest memory available on the GPU
- ❑ On Volta and later GPU architectures, the data exchange primitives can be used in thread-divergent branches
 - Threads can run the shuffle even when reaching different instances of the instruction, so long as they are mutually exclusive instances. E.g., this is valid:

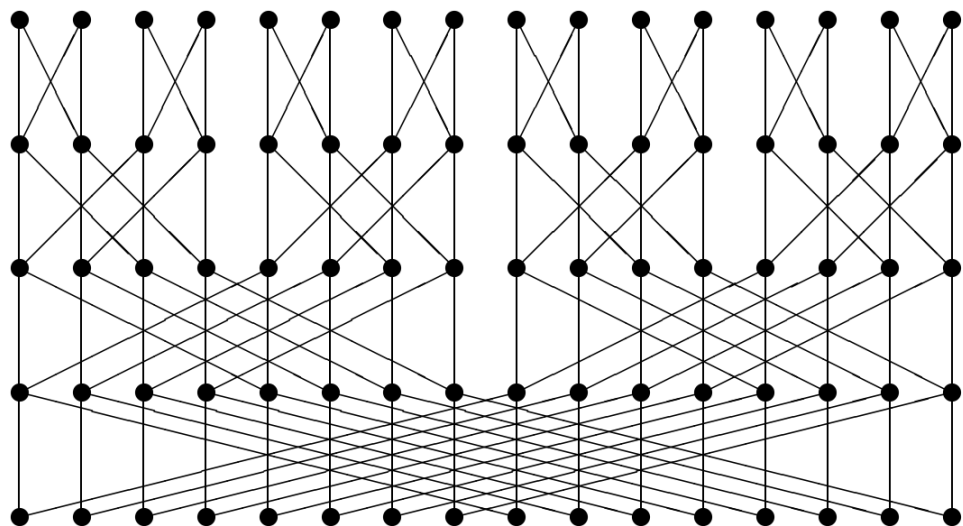
```
if (threadIdx.x % 2) {  
    val -= __shfl_sync(FULL_MASK, val, 0);  
} else {  
    val += __shfl_sync(FULL_MASK, val, 0);  
    // ...  
}
```

Extra: Warp Shuffles

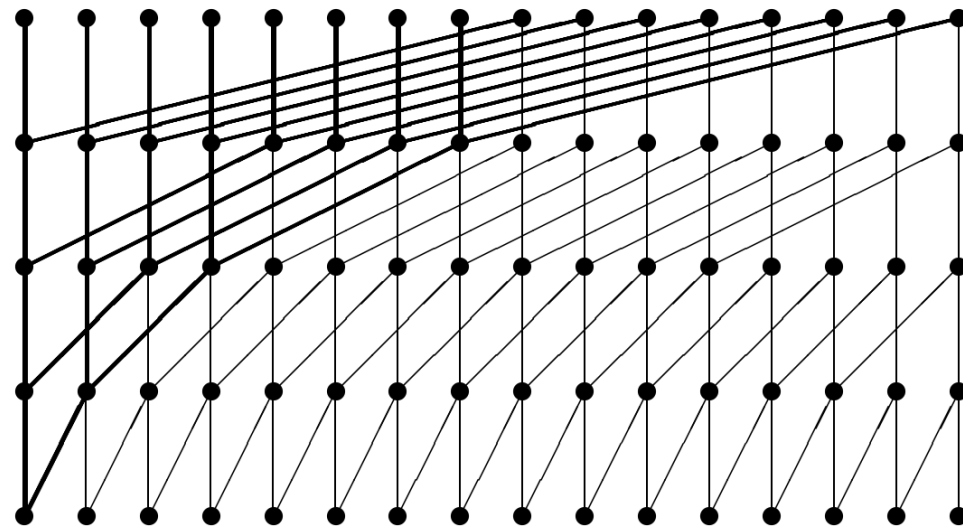
- ❑ Warp shuffles can speed up most collective operations (e.g. reduce, scan, ...) by suffling data between threads at the warp-level with SIMD execution
- ❑ This opens another opportunity for faster data access:
 - **Threads in the same warp $\Rightarrow$ shuffles (registers)**
 - Threads in the same block $\Rightarrow$ shared memory
 - Otherwise $\Rightarrow$ global memory (with atomics)
- ❑ Threads may only read data from another thread which is actively participating in the shuffle command. If the target thread is inactive, the retrieved value is undefined
 - This means you must be very careful with conditional code

Two Ways to Sum All Elements of a Warp

```
for (int i=1; i < warpSize; i*=2)
    value += __shfl_xor_sync(-1, value, i);
```



```
for (int i=warpSize/2; i>0; i=i/2)
    value += __shfl_down_sync(-1, value, i);
```



Reduce (array sum) with Warp Suffles

```
__global__ void sum_reduction(float* input, float* output) {
    __shared__ float warp_sums[32]; // assume 32 warps per block
    unsigned int g = blockIdx.x * blockDim.x * 2 + threadIdx.x;
    unsigned int t = threadIdx.x;
    float value = input[g] + input[g + blockDim.x];
    for (int offset = warpSize / 2; offset > 0; offset /= 2)
        value += __shfl_down_sync(0xffffffff, value, offset);

    int warp_id = t / warpSize;
    int lane_id = t % warpSize;
    if (lane_id == 0)
        warp_sums[warp_id] = value;
    __syncthreads();
    if (warp_id == 0) { // warp 0 reduces the results of all other warps
        value = (t < blockDim.x / warpSize) ? warp_sums[lane_id] : 0.0f;
        for (int offset = warpSize / 2; offset > 0; offset /= 2)
            value += __shfl_down_sync(0xffffffff, value, offset);
        if (lane_id == 0) // thread 0 writes to global memory
            atomicAdd(output, value);
    }
}
```

Credits & References

- ❑ Material first adapted by [Serena Curzel](#)
- ❑ Nvidia Deep Learning Institute, “Accelerated Computing” Teaching Kit
- ❑ D.B. Kirk and W.W. Hwu, “Programming Massively Parallel Processors. A hands-on approach”, 3rd edition, Morgan Kaufmann 2017
- ❑ Mark Harris, Parallel Prefix Sum with CUDA
https://developer.nvidia.com/gpugems/GPUGems3/gpugems3_ch39.html
- ❑ Nvidia Technical Blog “Using CUDA Warp-Level Primitives”
<https://developer.nvidia.com/blog/using-cuda-warp-level-primitives/>
- ❑ Mike Giles “Course on CUDA Programming on NVIDIA GPUs”
<https://people.maths.ox.ac.uk/~gilesm/cuda/>
- ❑ Disclaimer: all code in these slides “forgot about” boundary conditions...